



tutorial_communication,
a tutorial on how to instrument a code with OASIS3-MCT API calls
August 2023

Introduction

This directory contains the files of a tutorial on how to instrument a code with calls to the OASIS3-MCT library in order to couple it with other components. The tutorial involves two toy model codes, `ocean.F90` and `atmos.F90`, which give you a practical example on the use of the OASIS3-MCT API (Application Program Interface) routines. Toy models are skeleton programs that do not contain any real physics or dynamics but that can reproduce real exchanges of coupling fields.

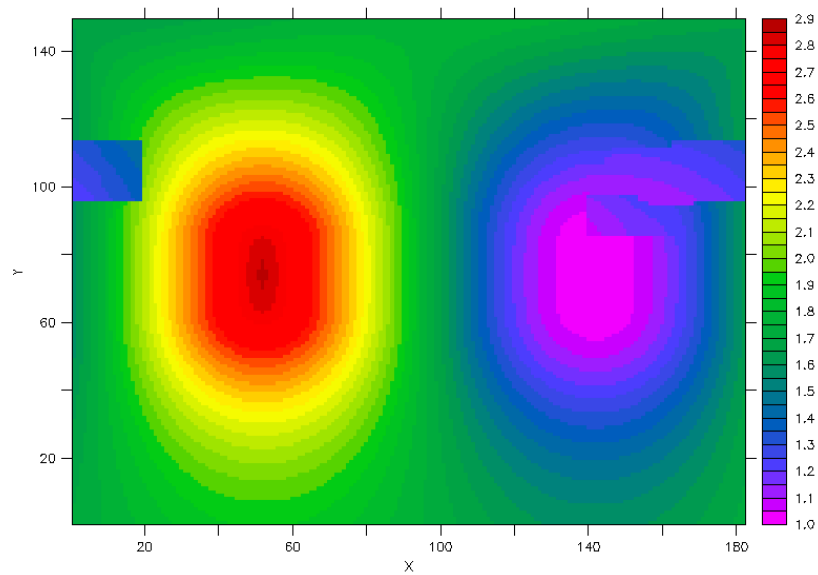
This tutorial is extracted from the Short Private Online Course (SPOC) on “Code Coupling with OASIS3-MCT” developed in the framework of the ESiWACE Centre of Excellence. This SPOC is composed of videos, quizzes and hands-on. If you are interested in attending the SPOC, please visit the online training section of CERFACS web site at <https://cerfacs.fr/online-training/>. Videos and quizzes were also extracted from the SPOC and are available as Open Education Resources (OER) material at <https://www.oercommons.org/courseware/lesson/85340>.

OASIS3-MCT is a coupling library that can be used to exchange information, the so-called coupling fields, between different codes or even within one same code. Internally, OASIS3-MCT uses the Model Coupling Toolkit (MCT) developed by the Argonne National Laboratory. Usually, the codes to be coupled represent different parts of the Earth System, e.g. the ocean or the atmosphere. The codes can be sequential, i.e. run on only one process, or parallel, i.e. run on more than one process. Each code must be compiled and linked with the OASIS3-MCT library. During the simulation, the OASIS3-MCT library performs the exchange of coupling fields between the different executables (i.e. the compiled codes). When the code is parallel, each process can send or receive its part of the coupling field (i.e. there is no need to gather the whole coupling field on one process). When the OASIS3-MCT library performs the exchanges, it can apply different transformations on the coupling fields, for example spatial regridding (i.e. transforming the coupling field from the grid of the source code to the grid of the target code).

The coupling exchanges implemented in tutorial_communication coupled model

The tutorial implement basic coupling exchanges between the toy models `ocean.F90` and `atmos.F90`. In their original form, `ocean` and `atmos` run for 4 hours (14400 seconds); `ocean` has a timestep of 3600 sec and `atmos` a timestep of 1800 sec.

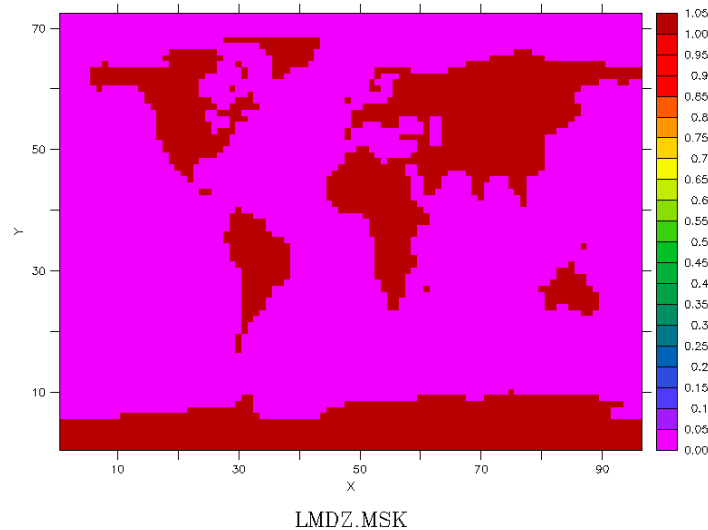
The grid of `ocean` has 182x149 grid points globally but `ocean` can be parallel so each `ocean` process defines its local partition and reads only its part of the grid in the file `ocean_mesh.nc`. Then each process allocates two local arrays `field_send_ocean`, a coupling field to send, and `field_rcv_ocean`, the coupling field to receive. The coupling field `field_send_ocean` is function of the longitude and the latitude of the grid points and of the timestep number, so it varies at each timestep. At the first timestep, `field_send_ocean` looks like this (in the 182x149 grid point space):



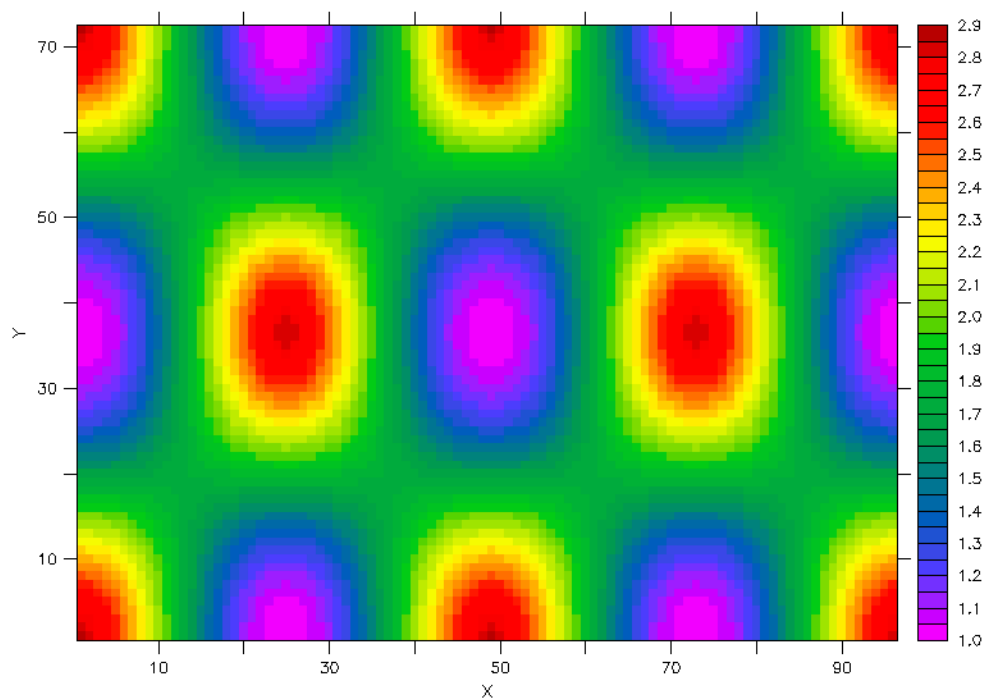
Note that the square-like patterns are due to the distortion of the grid over the continents.

The grid of `atmos` has 96 x 72 grid points globally but, as `ocean`, `atmos` is parallel so each `atmos` process defines its local partition and reads only its part of the grid in the file `atmos_mesh.nc`. The next figure illustrates the sea-land mask on the `atmos` grid.

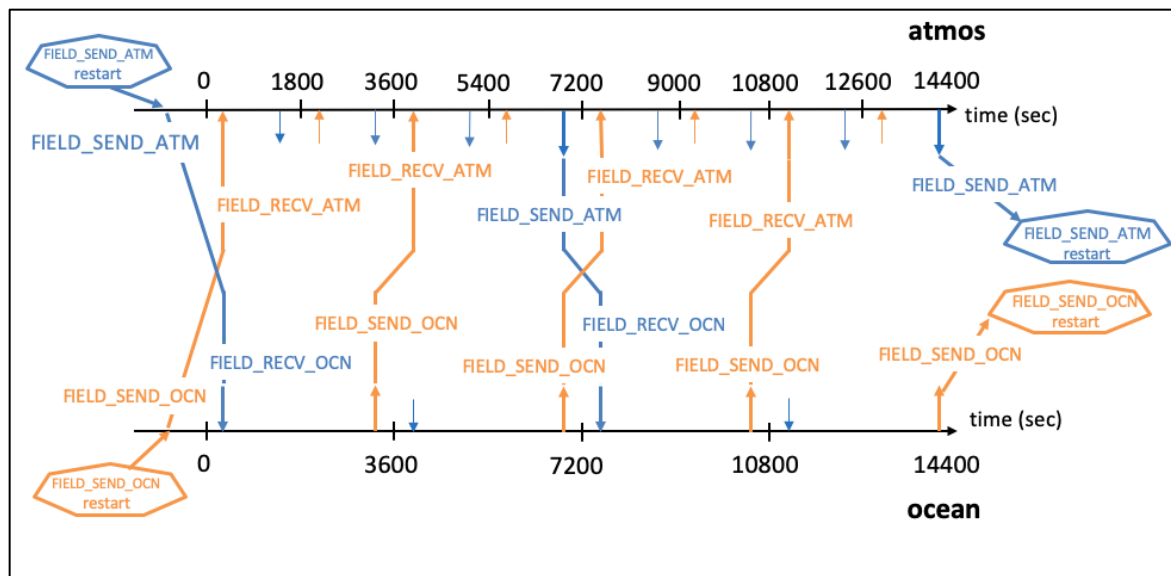
Following OASIS3-MCTconvention, masked cells have a value of 1 (red) and non-masked cells a value of 0.



Then each process allocates two local arrays `field_send_atmos`, the coupling field to send and `field_rcv_atmos` the coupling field to receive. The coupling field `field_send_atmos` is defined with a function that varies with the longitude and the latitude faster than in ocean and it also varies in time. At the first timestep, `field_send_atmos` looks like this (in the 96 x 72 grid point space):



The coupling exchanges implemented are illustrated here. The ocean sends its array `field_send_ocean` (with symbolic name `FIELD_SEND_OCEAN`) to `atmos` which receives it in the array `field_rcv_atmos` (with symbolic name `FIELD_RECV_ATMOS`); the coupling period is 3600 sec. In return, `atmos` sends its array `field_send_atmos` (with symbolic name `FIELD_SEND_ATMOS`), to `ocean` which receives it in the array `field_rcv_ocean` (with symbolic name `FIELD_RECV_OCEAN`); the coupling period is 1800 sec.



Compiling and running tutorial toy models

Following the rules of OASIS3-MCT compiling and run environment effective since December 2022, two environment variables `$OASIS_COUPLE` and `$OASIS_ENV` must be defined. `$OASIS_COUPLE` gives the path of the sources of the coupler (`/lib`, `/util`, `/examples`, etc. directories). `$OASIS_ENV` gives the extension of the header `make.${OASIS_ENV}` from directory `${OASIS_COUPLE}/util/make_dir` to use.

Typing `./oasis_test_build.sh` compiles OASIS3-MCT itself and executables `atmos` and `ocean`. This script invokes the Makefile in the tutorial directory (with `"make"`), which automatically uses the OASIS3-MCT header `make.${OASIS_ENV}`, via the inclusion of `$(OASIS_COUPLE)/util/make_dir/make.inc` – see the first line in the Makefile.

Then the local script `oasis_test_run.sh` can be used to run the tutorial coupled model with `./oasis_test_run.sh param_tutorial_communication_test01`, where the file `param_tutorial_communication_test01` configures the test specifying :

- the name of the executables `USER_EXE1` and `USER_EXE2`,
- the number of processors for each model `USER_NPEXE1` and `USER_NPEXE2`,
- the name and directory of the namcouple, `USER_NAMCOUPLE` and `USER_NAMLOC`
- the name and directory of the Makefile, `USER_MAKEFILE` and `USER_MAKELOC`
- the remapping file directory `USER_RMPLOC`,
- the directory containing OASIS3-MCT auxiliary files `grids.nc`, `areas.nc` and `masks.nc`, `USER_AUXLOC` (not defined here as `atmos` and `ocean` create them using `oasis_write_grid`, `oasis_write_corner`, `oasis_write_mask` API routines),
- the directory containing the coupling restart files `USER_RSTLOC`,
- the directory that contains the file defining the model grid `USER_MESHLOC`.

Note that the namcouple file from `$USER_NAMLOC=data_tutorial` specifies that a coupling field with symbolic name `FIELD_SEND_OCN` has to be matched with a coupling field with symbolic name `FIELD_RECV_ATM` and that the mapping between the two fields has to use the pre-existing weight and addresses file

my_remapping_file_bilinear.nc. It also specifies that a coupling field with symbolic name FIELD_SEND_ATM has to be matched with a coupling field with symbolic name FIELD_RECV_OCN and that the interpolation to transform the field has to be the SCRIPR/BILINEAR one (i.e. the weight and addresses file used for the interpolation will be calculated in the initialisation phase with the SCRIP library).

The results of the tutorial coupled model should now be in your subdirectory TESTS/work_tutorial_communication_namcouple_Makefile_atmos_1_ocean_1. The file nout.000000, written by the master process of one model, contains the information read in the configuration file namcouple. The OASIS3-MCT debug files debug.01.000000 and debug.02.000000 are written respectively by the ocean and atmos process respectively. The level of debug information in these files corresponds to the value of the first number on the line below \$NLOGPRT in the namcouple.

If your run was successful, these debug files should all end with :

```
-- EXIT (oasis_terminate)
```

The tutorial coupled model can be run on different numbers of processes for atmos and ocean by changing USER_NPEXE1 and USER_NPEXE2 in the parameter file. For example, if one puts USER_NPEXE1=4 and USER_NPEXE2=4, each model will run on 4 processes. You can experiment this by typing “./oasis_test_run.sh param_tutorial_communication_test02” and the results of the run will be in directory

TESTS/work_tutorial_communication_namcouple_Makefile_atmos_4_ocean_4

In that case, OASIS3-MCT debug files will be debug.01.00000? and debug.02.00000? where ? goes from 0 to 3, one for each process. The files debug.01.000000 and debug.02.000000 should end as above but the other debug files will simply end with :

```
-- EXIT (oasis_terminate)
```

If you need further help, do not hesitate to consult the OER material available at <https://www.oercommons.org/courseware/lesson/85340> and/or to follow the SPOC on “Code Coupling with OASIS3-MCT” (see <https://cerfacs.fr/online-training/>). We are also available at oasishelp@cerfacs.fr to answer questions you may have.

Have fun with OASIS3-MCT !

The OASIS team